Agents and Reliability

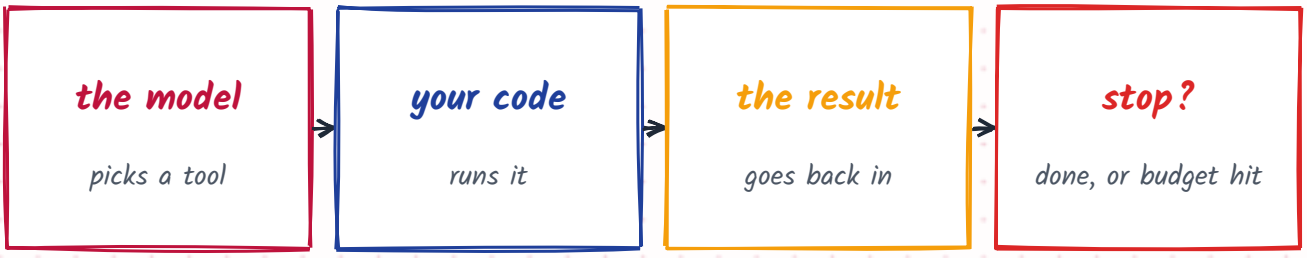
Lesson 9 of 12 · The agent loop, MCP, and how not to refund twice.

In this lesson → ① The agent loop ② What loops cost ③ Prompt caching ④ MCP and A2A ⑤ The crash experiment ⑥ Jitter and breakers ⑦ Interview Q&A

Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 What an agent is

An **agent** = a model that uses tools in a loop, with memory and stop conditions (the classic recipe is called ReAct: think, act, observe, repeat). A chat answers once; an agent keeps going.



the loop repeats until the model is done or a limit stops it

```
messages = [system, user_task]
for step in range(MAX_STEPS):        # stop condition 1: a step budget
    reply = model(messages, tools)
    if reply.is_final: return reply.text   # stop condition 2: the model is done
    for call in reply.tool_calls:
        if needs_approval(call): wait_for_human(call)
        messages.append(run_tool(call))    # sandbox, timeout, errors returned
raise StepBudgetExceeded             # fail loudly
```

Sketch from the notebook. The model only writes a tool name and JSON arguments; **your code** runs it.

2 Tools, and workflows vs agents

→ Document tools like docs for a new colleague (examples, edge cases)
→ Return errors as results so the model self-corrects
→ Make mistakes impossible (absolute paths only made a whole error class vanish)

Pattern	Use when
chain	fixed steps; trade speed for accuracy
route	distinct categories of request
parallel (split / vote)	independent parts, or compare several runs
orchestrator-workers	subtasks you cannot predict
writer + critic	clear quality criteria

Start simple
A workflow follows a path you wrote; an agent decides its own. Start as a workflow: cheaper, faster, easier to test.

3 Many agents, and memory

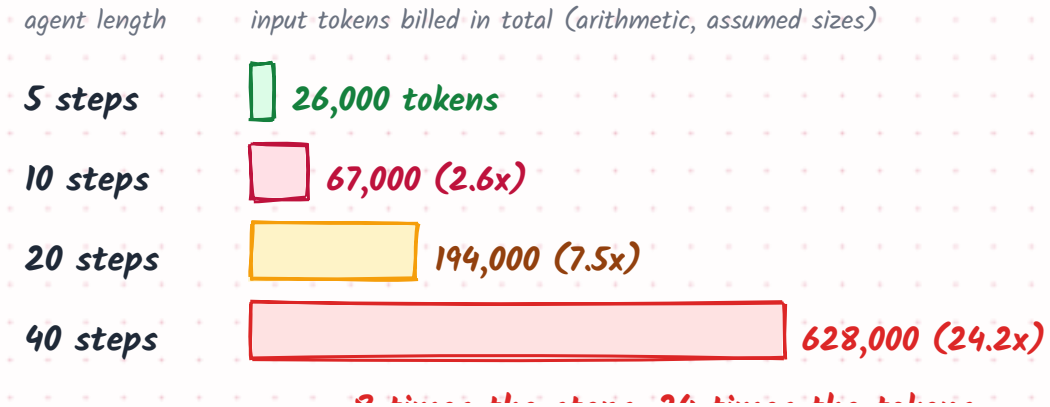
Setup	Tokens vs a chat
chat	1x
single agent	about 4x
multi-agent	about 15x

Reported by Anthropic for its research system (notebook). Pay 15x only for broad, parallel, high-value work. Poor fits: shared context, tight dependencies, most coding.

→ **Short-term memory**: this thread, checkpointed so a run can resume
→ **Long-term memory**: across sessions
→ Treat context as a **budget**: every kept token is paid for on every later step

4 Input tokens grow with steps squared

An agent re-sends its history each step. Assumed sizes: system prompt + tools 4,000 tokens; each step adds 100 (reply) + 500 (tool result). **input at step n = 4,000 + 600 × (n − 1)**

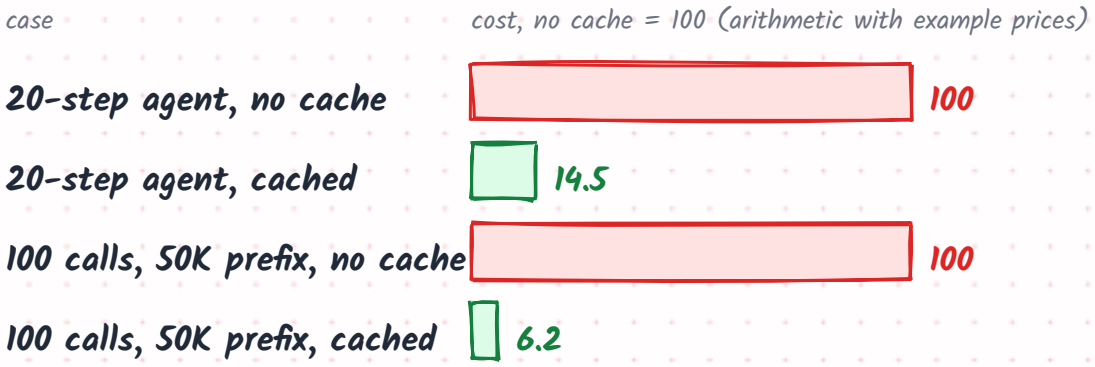


Steps	Last step input	Total input tokens	Cost vs 5 steps
5	6,400	26,000	1.0x
10	9,400	67,000	2.6x
20	15,400	194,000	7.5x
40	27,400	628,000	**24.2x**

ARITHMETIC with assumed sizes. Script: l09_agent_cost.py.

5 Prompt caching

> The provider keeps the work for an identical prompt start and charges about a tenth to read it back. Example prices (a Sonnet-class model, notebook): input $2.00, cache write $2.50, cache read $0.10 per million tokens.



Case	No cache	Cached	Saving
20-step agent (above)	$0.388	$0.056	85%
100 calls, 50,000-token shared prefix	$10.00	$0.620	94%

- ✓ Stable content first, volatile last; no timestamps or user names at the top
- ✓ Invalidation order: tools, then system, then messages: changing a tool loses everything
- ✓ Prompts under a minimum length are silently not cached; caches are per model, so falling back to another model loses it

6 Semantic caching: careful

- Lossy: "cancel my order" looks like "cancel my subscription"
- An answer built from one user's documents must never be served to another

Use only for generic FAQ traffic, with a strict similarity threshold, short lifetimes, and keys per tenant.

7 MCP: a standard plug for tools



MCP = agent to tools · A2A = agent to agent

→ **Host** (your app) runs one **client** per **server**; JSON-RPC 2.0

→ A server offers **tools** (the model controls), **resources** (the app controls), **prompts** (the person controls)

→ Transports: a local process (stdio) or Streamable HTTP; auth with OAuth 2.1

8 MCP, 2026-07-28: stateless

- → No initialize handshake and no session ID: every request carries its version, identity and capabilities
- → Any instance behind a load balancer can answer: run a server as a plain Lambda / Cloud Run function
- → Mcp-Method and Mcp-Name headers let gateways route and limit per tool; lists are cacheable

9 A2A: agent to agent

Protocol	Connects
MCP	an agent calls a server's tools
A2A (v1.0, March 2026)	an agent delegates a task to another agent, found through its "Agent Card"

10 Long tasks: async and durable

- → API returns **202 + a task ID**; the task goes on a queue
- → A durable workflow engine (Temporal, Step Functions, Durable Functions, LangGraph) runs the loop; each call is a retried step
- → Code runs in a per-task sandbox; state is checkpointed to a database; progress by SSE or webhook

11 Experiment: the crash

Item	Value
loop	working: step budget, tool errors returned, approval gate, checkpoint after every step
"model"	a **scripted stand-in**: no language model is called
task	look up, policy, eligibility, draft, approval, **refund**, **email**, update ticket (9 tool steps)
failure	the process dies just before the email step (step 8)



crash just before the email step

Recovery	Tool calls	Refunds	Emails
restart, no keys	14	**2**	1
restart + idempotency keys	14	1	1
resume, no keys	8	1	1
resume + keys	8	1	1

Script: l09_agent_loop.py.

12 Four rules for long runs

- ✓ **Checkpoint after every step**: a crash at step 37 resumes at 37
- ✓ **Idempotency key** on every tool with a side effect: a replay cannot charge or email twice
- ✓ **Approvals** are a durable pause, not a blocked process
- ✓ Running tasks finish on the version they started with

13 Where agent code runs

Sandbox	Strength
micro-VM (Firecracker)	strongest: its own kernel
gVisor	middle: intercepts system calls
container	weakest: shares the host kernel, trusted code only

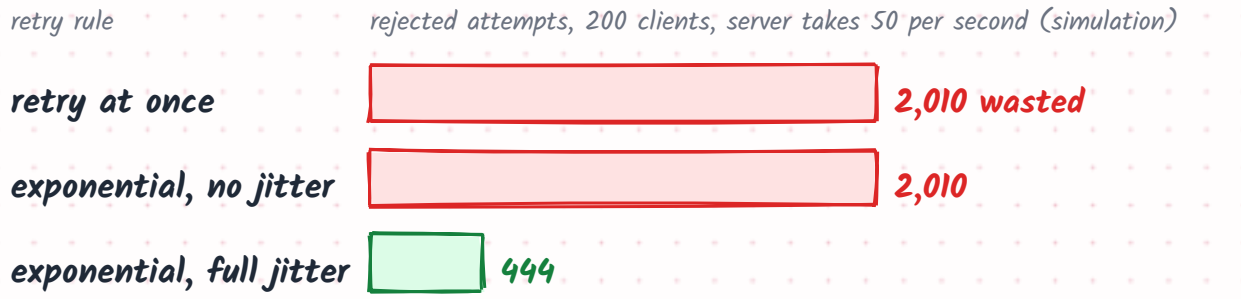
Untrusted code: micro-VM, time and memory limits, controlled outbound traffic, **no secrets inside**.

14 Reliability in layers

Layer	Does
SDK	jittered retries
gateway	circuit breaker, fallback, quotas
workflow engine	durable retries
app	streaming, idempotency keys

15 Experiment: the retry storm

200 clients all call at second 0; the server accepts 50 requests per second **(assumed)**; each client gives up after 12 tries. SIMULATION in 0.1 s ticks.



Retry rule	Attempts	Rejected	Last success	Gave up
retry at once (0.1 s)	2,070	2,010	1.1 s	**140**
exponential, no jitter	2,070	2,010	**211.0 s**	140
exponential, full jitter	644	**444**	**9.9 s**	0

retry at once

140 clients gave up

no jitter

last client at 211 s

full jitter

last client at 9.9 s

random waits spread the retries out

16 Retry rules

> **wait = random(0, min(cap, base × 2^attempt))** ("full jitter"). If the server sends Retry-After, obey it first.

```
import random, time
RETRYABLE = {429, 500, 503, 529}
def call_with_backoff(send, base=0.5, cap=30.0, max_attempts=6):
    for attempt in range(max_attempts):
        resp = send()
        if resp.status not in RETRYABLE: return resp
        ra = resp.headers.get("retry-after")
        time.sleep(float(ra) if ra else random.uniform(0, min(cap, base * 2 ** attempt)))
    raise RuntimeError("gave up")
```

From the notebook. Retry 429, 500, 503, 529; never blindly retry a 400. Script: l09_retry_breaker.py.

17 Circuit breakers



success closes it · failure opens it again

policy	calls that hit a backend that is down for 30 s (simulation)
no breaker	**600 calls to a dead backend**
circuit breaker	**7**

fail fast while it is down

*SIMULATION: a backend down for 30 s, 20 requests per second for 60 s. No breaker: 1,200 calls, **600** hit the dead backend. Breaker (opens after 5 failures, 10 s cooldown, one trial): 607 calls, **7** hit it.*

18 Fallbacks and gateways

- Fallback order: **same model, another region** first; then another model (behaviour changes; the prompt cache is lost)
- An AI gateway: one API, key vault, per-team quotas, routing, fallbacks, metrics. Extend what you already run
- Streaming for fast first tokens and no idle timeouts; structured outputs follow a schema, but **valid is not correct**

19 One stack, three clouds

Layer	What lives there
edge + API	firewall, API gateway
agent runtime	your loop with a durable workflow engine
AI gateway	quotas, fallback, guardrails
beside them	retrieval, state, queues, sandboxed tool servers, identity, tracing

My synthesis, not a vendor table. AWS: Bedrock + AgentCore; Google: Gemini + Agent Engine; Azure: Foundry Agent Service.

20 ShopNest Copilot: agent plan

Piece	Plan
shape	a workflow first; an agent only if the evals say so
refund tool	human approval + idempotency key + checkpoint
tools	served over MCP
calls	stable prefix first, through a gateway, jitter, breaker

21 Words to keep

Word	Meaning
Idempotency key	a label that makes a repeated action happen only once
Checkpoint	saved progress to resume from
Jitter	a random delay in retries
Circuit breaker	a switch that stops calls to a broken service

22 Interview corner

Q. How do you make a long-running agent safe to retry?

A. Checkpoint after every step, resume rather than restart, idempotency keys on tools with side effects. In my test a restart refunded twice and a resume once.

Q. Why add jitter to retries?

A. Without it clients retry in waves. 200 clients: full jitter finished in 9.9 s against 211 s, with a quarter of the wasted attempts (444 vs 2,010).

Q. Why do long agent loops get expensive?

A. The history is re-sent every step, so total input tokens grow with the square of the steps: 8x the steps cost 24x the tokens. Prompt caching cuts it by 85 to 94% in my examples.

Q. MCP or A2A?

A. MCP: an agent calls a server's tools. A2A: an agent delegates to another agent.

23 Interview questions

"Workflow or agent: how do you decide?"

Start as a workflow: a path you wrote is cheaper, faster and easier to test. Use an agent only when the subtasks cannot be predicted and the evals show it helps. Reported tokens: a single agent about 4x a chat, multi-agent about 15x.

Trap → Building a multi-agent system for tightly dependent work such as most coding.

"Why do agent costs grow faster than the number of steps?"

The whole history is re-sent every step. With my assumed sizes, 40 steps used 628,000 input tokens: 24.2x the cost of 5 steps for 8x the steps. Prompt caching cut a 20-step run by 85%.

Trap → Putting changing text, like a timestamp, at the top: it breaks the cache.

"How do you make tool calls safe to retry?"

An idempotency key on every tool with a side effect, and a checkpoint after every step so a crash resumes instead of restarting. In my test a restart without keys refunded twice; a resume refunded once.

Trap → Retrying a payment or email tool with no idempotency key.

"What is MCP, and how is it different from A2A?"

MCP is a standard plug for tools: a host runs one client per server over JSON-RPC 2.0, and a server offers tools, resources and prompts. A2A lets an agent delegate a whole task to another agent, found through its Agent Card.

Trap → Using A2A when all you need is to call a tool.

"How do you design retries for an overloaded API?"

Exponential backoff with full jitter, and obey Retry-After first. With 200 clients in my simulation, full jitter finished in 9.9 s with 444 rejections; no jitter took 211 s with 2,010.

Trap → Blindly retrying a 400. Retry 429, 500, 503, 529.

"Where should agent-generated code run?"

In a per-task micro-VM (Firecracker, its own kernel) with time and memory limits, controlled outbound traffic and no secrets inside. gVisor is the middle option; a plain container shares the host kernel.

Trap → Running untrusted code in a plain container.

24 Your turn

109_agent_loop.py (remove the idempotency key and watch the refund happen twice), **109_retry_breaker.py**, **109_agent_cost.py**.

⚡ Revise in 60 seconds

- → agent = model + tools + loop; workflow first
- → 8x steps = **24x** tokens; caching saves 85 to 94%
- → crash: restart **2 refunds**, resume or key **1**
- → MCP = agent to tools · A2A = agent to agent
- → jitter: 211 s → **9.9 s**, 140 gave up → 0
- → breaker: 600 → **7** calls to a dead backend
- → fallback: same model, other region first
- → valid output is not correct output

Next lesson → CAI-10
Security and governance: identity, prompt injection, least privilege
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**